

PYTHON PROGRAMMING INTERNSHIP REPORT

STUDENT MANAGEMENT SYSTEM

INTERNSHIP PROJECT REPORT

SUBMITTED BY

RIYA LAJIKUMAR

BTECH COMPUTER SCIENCE & ENGINEERING

SUBMITTED TO

SYNTECXHUB– PYTHON PROGRAMMING TASK 2

INTERNSHIP AT

SYNTECXHUB

COURSE

PYTHON PROGRAMMING

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Syntecxhub for providing me with the opportunity to undertake this project titled "**Student Management System**." This project has been a valuable learning experience that allowed me to apply theoretical concepts of Python programming to a real-world application. Through this project, I gained practical knowledge of Object-Oriented Programming (OOP), file handling, data validation, and software development practices.

I would also like to thank my mentors, faculty members, and friends for their continuous guidance, encouragement, and support throughout the project development process. Their valuable suggestions helped me overcome challenges and improve the quality of the project. Finally, I extend my heartfelt gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

ABSTRACT

The Student Management System is a Python-based Command Line Interface (CLI) application developed to manage student records efficiently and systematically. Educational institutions require a reliable mechanism for storing, updating, and retrieving student information. Manual methods of record maintenance are often inefficient, time-consuming, and prone to errors. This project aims to address these challenges by providing an automated solution for student record management.

The system allows users to perform various operations such as adding new student records, updating existing information, deleting records, and viewing all available student data. The project utilizes Object-Oriented Programming principles, including classes, objects, constructors, and methods, to create a modular and maintainable application structure. Student records are stored permanently using JSON file storage, ensuring that data remains available even after the program is terminated.

The project demonstrates the practical implementation of Python programming concepts such as file handling, exception handling, data validation, and formatted console output. It serves as an excellent example of how software applications can simplify administrative tasks and improve data management efficiency in educational environments.

TABLE OF CONTENTS

CONTENTS

1. ACKNOWLEDGEMENT.....	i
2. ABSTRACT.....	ii
3. TABLE OF CONTENTS.....	iii
4. INTRODUCTION.....	1
5. LITERATURE SURVEY.....	1
6. PROBLEM STATEMENT.....	2
7. PROJECT OBJECTIVES.....	2
8. SCOPE OF THE PROJECT.....	3
9. SYSTEM ANALYSIS.....	3
10. SYSTEM DESIGN.....	4
11. HARDWARE REQUIREMENTS.....	5
12. SOFTWARE REQUIREMENTS.....	5
13. TECHNOLOGIES USED.....	6
14. ALGORITHM.....	7
15. FLOWCHART.....	8
16. SOURCE CODE.....	9
17. SAMPLE OUTPUTS.....	13
18. TESTING AND VALIDATION.....	17
19. ADVANTAGES.....	18
20. LIMITATIONS.....	18
21. FUTURE ENHANCEMENTS.....	18
22. CONCLUSION.....	19

1.INTRODUCTION

In today's digital world, educational institutions generate and manage a large volume of student information. Maintaining student records manually using notebooks, registers, or spreadsheets can be inefficient and may lead to data inconsistency, duplication, and errors. As the number of students increases, it becomes increasingly difficult to organize and retrieve information efficiently. Therefore, there is a need for an automated system that can simplify and streamline student record management.

The Student Management System is designed to provide an efficient and user-friendly solution for handling student information. The application is developed using Python and follows Object-Oriented Programming principles. The system allows users to add, update, delete, and view student records through a menu-driven interface. Student information is stored in a JSON file, enabling permanent storage and retrieval of data.

The project aims to demonstrate fundamental software development concepts such as modular programming, file handling, data validation, and object-oriented design. It provides a practical example of how programming can be used to automate administrative tasks and improve productivity. The system is designed to be simple, reliable, and easy to maintain, making it suitable for academic learning and small-scale educational environments.

2.LITERATURE SURVEY

Student information management has evolved significantly over the years. Traditional methods relied on paper-based record keeping, where student details were maintained manually in registers and files. While these methods were simple to implement, they often resulted in challenges such as data loss, duplication, and difficulty in retrieving information quickly. As educational institutions expanded, the limitations of manual record management became increasingly apparent.

With advancements in information technology, computer-based management systems were introduced to improve efficiency and accuracy. Modern Student Management Systems utilize databases, web technologies, and cloud platforms to provide comprehensive solutions for handling student information. These systems support features such as attendance tracking, examination management, report generation, and communication tools.

Python has emerged as one of the most popular programming languages for developing management systems due to its simplicity, readability, and extensive library support. JSON-based data storage provides a lightweight and efficient method for maintaining records without requiring a dedicated database system. The Student Management System developed in this project combines Python programming and JSON storage to create a practical and easy-to-use solution for student record management.

3.PROBLEM STATEMENT

Educational institutions need an effective mechanism for managing student records. Traditional manual methods of maintaining student information are often inefficient, time-consuming, and prone to human errors. Retrieving specific records from large collections of data can be difficult, and updating information frequently requires significant effort.

Another challenge associated with manual systems is the risk of data duplication and inconsistency. Multiple entries for the same student may occur, leading to confusion and inaccurate records. Additionally, maintaining paper-based records increases the possibility of data loss due to physical damage or misplacement.

The objective of this project is to develop a Student Management System that automates the process of managing student information. The system ensures that each student has a unique identification number, supports efficient record operations, and provides permanent storage through JSON files. By automating these processes, the project aims to improve accuracy, efficiency, and reliability in student record management.

4.PROJECT OBJECTIVES

The primary objective of the Student Management System is to develop a reliable and user-friendly application for managing student records. The project aims to automate common administrative tasks and provide a structured approach to handling student information.

Specific objectives include:

- To design and implement a menu-driven Student Management System using Python.
- To demonstrate the use of Object-Oriented Programming concepts such as classes, objects, constructors, and methods.
- To provide functionalities for adding, updating, deleting, and viewing student records.
- To ensure data integrity through validation mechanisms, including unique student identification numbers.
- To implement permanent data storage using JSON files.
- To improve efficiency and reduce manual effort in managing student information.
- To provide formatted and user-friendly console output for better usability.

5.SCOPE OF THE PROJECT

The Student Management System is designed to simplify the process of managing student information within educational institutions. The project provides a centralized and organized method for storing student records and performing common administrative operations. The application allows users to add new students, update existing student information, delete records, and display all stored records through a menu-driven interface.

The scope of the project extends to educational institutions such as schools, colleges, tuition centers, and training organizations that require an efficient method of maintaining student information. The system helps reduce manual work and improves the speed and accuracy of data management. Although the current version focuses on basic student record operations, it can serve as a foundation for more advanced systems involving attendance management, examination processing, report generation, and database connectivity.

The project is particularly useful for learning software development concepts such as Object-Oriented Programming, file handling, and data validation. It provides practical exposure to designing and implementing a real-world application using Python.

6.SYSTEM ANALYSIS

System analysis is the process of studying the existing system and identifying the requirements for a new system. It helps in understanding the limitations of current methods and determining how a computerized solution can improve efficiency.

- **Existing System:** In many educational institutions, student records are maintained manually using paper registers or spreadsheet applications. Such methods often require significant effort to update and maintain information. Searching for student records becomes increasingly difficult as the number of students grows. The existing system suffers from several limitations including data duplication, inaccurate record maintenance, slow retrieval of information, and a high risk of data loss due to human error. Manual record management also requires additional storage space and consumes valuable administrative time.
- **Proposed System:** The proposed Student Management System provides an automated solution for managing student records. The application stores student information electronically and enables quick access to records. The use of JSON file storage ensures that data remains permanently available and can be retrieved whenever needed. The proposed system improves efficiency, reduces manual workload, prevents duplicate student IDs, and provides accurate data management. The system also offers a user-friendly interface that simplifies record operations for users.

7.SYSTEM DESIGN

System design defines how the application is structured and how its components interact to achieve the desired functionality.

a) Input Design:

The system accepts the following inputs from the user:

- Student ID
- Student Name
- Student Grade

These inputs are validated before being stored in the system to ensure data accuracy.

b) Process Design:

The application processes user requests through the following operations:

- Add Student Record
- Update Student Information
- Delete Student Record
- Display Student Records
- Save and Load Records

c) Output Design

The system produces outputs in the form of:

- Formatted student record tables
- Success messages
- Error notifications
- Validation alerts

The output is displayed clearly in the console to ensure easy readability and understanding.

8.HARDWARE REQUIREMENTS

The Student Management System requires only basic hardware resources because it is a lightweight command-line application.

Minimum Hardware Requirements:

- Processor: Intel Core i3
- RAM: 4 GB
- Hard Disk: 500 MB free space
- Monitor
- Keyboard and Mouse

Recommended Hardware Requirements:

- Processor: Intel Core i5 or above
- RAM: 8 GB
- SSD Storage
- Full HD Monitor

These requirements ensure smooth execution of the application without performance issues.

9.SOFTWARE REQUIREMENTS

The software requirements necessary for developing and executing the project are listed below:

Operating System:

- Windows 10
- Windows 11
- Linux
- macOS

Development Tools:

- Visual Studio Code
- Python Interpreter

Programming Language:

- Python 3.x

Libraries and Modules:

- JSON Module
- OS Module

Version Control:

- GitHub

The software stack used in this project is free, open-source, and easy to install.

10. TECHNOLOGIES USED

Python:

Python is the primary programming language used for developing the Student Management System. It is known for its simplicity, readability, and extensive support for software development.

JSON:

JSON (JavaScript Object Notation) is used for storing student records permanently. JSON provides a lightweight and structured format for data storage.

Object-Oriented Programming (OOP):

The project uses OOP concepts such as:

- Classes
- Objects
- Constructors
- Methods
- Encapsulation

These concepts improve code organization and maintainability.

Visual Studio Code:

Visual Studio Code serves as the Integrated Development Environment (IDE) used for writing, testing, and debugging the application.

GitHub:

GitHub is used for project version control and source code management.

11.ALGORITHM

Step 1: Start the application.

Step 2: Load student records from the JSON file.

Step 3: Display the main menu.

Step 4: Read user choice.

Step 5: If the user selects Add Student:

- Enter Student ID.
- Check whether the ID already exists.
- If unique, collect Name and Grade.
- Save record.

Step 6: If the user selects Update Student:

- Enter Student ID.
- Search the record.
- Update information.
- Save changes.

Step 7: If the user selects Delete Student:

- Enter Student ID.
- Locate the record.
- Remove it from storage.
- Save changes.

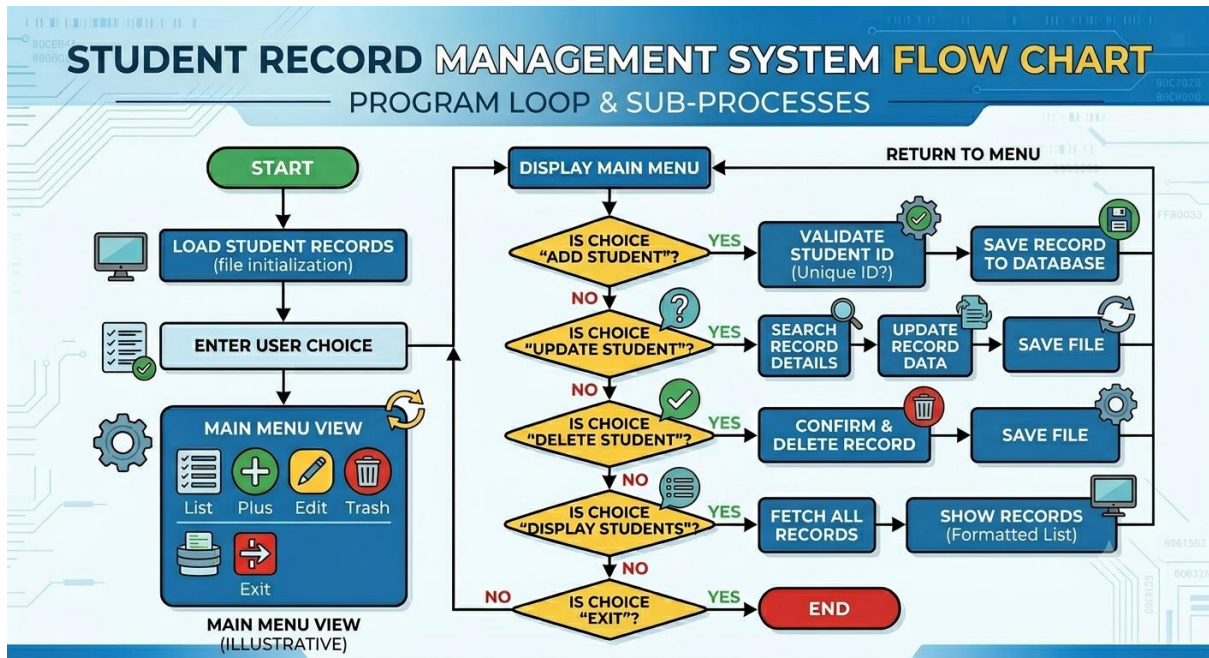
Step 8: If the user selects List Students:

- Retrieve all records.
- Display them in formatted form.

Step 9: Repeat menu operations until Exit is selected.

Step 10: End the program.

12.FLOWCHART



- This flowchart illustrates the functional architecture of a **Student Record Management System** driven by a standard program loop.
- The execution begins at the **Start** node, which immediately triggers the initialization process to load existing student data from the files.
- Once the data is loaded, the program enters the **Main Menu View** where it prompts the user to enter their desired operational choice.
- The user's input is passed into a vertical chain of **Decision Diamonds** that evaluate conditional logic to route the system's behavior.
- Depending on the selection, the system branches horizontally to execute core **CRUD operations**, such as validating a new student ID, updating existing data, or deleting a record.
- After any of these data-handling processes or a display request is completed, a feedback loop dynamically routes the program flow back to the **Main Menu**.
- The continuous cycle safely terminates only when the user selects the **Exit** option, directing the logic path to the final **End** node.

13.SOURCE CODE

```
import json
import os

FILE_NAME = "students.json"

class Student:
    def __init__(self, student_id, name, grade):
        self.student_id = student_id
        self.name = name
        self.grade = grade

    def to_dict(self):
        return {
            "id": self.student_id,
            "name": self.name,
            "grade": self.grade
        }

class StudentManager:
    def __init__(self):
        self.students = []
        self.load_students()

    def load_students(self):
        if os.path.exists(FILE_NAME):
            with open(FILE_NAME, "r") as file:
                data = json.load(file)
                self.students = data
```

```

def save_students(self):
    with open(FILE_NAME, "w") as file:
        json.dump(self.students, file, indent=4)

def add_student(self):
    student_id = input("Enter Student ID: ")

    for student in self.students:
        if student["id"] == student_id:
            print("Student ID already exists!")
            return

    name = input("Enter Name: ")
    grade = input("Enter Grade: ")
    student = Student(student_id, name, grade)
    self.students.append(student.to_dict())
    self.save_students()
    print("Student Added Successfully")

def update_student(self):
    student_id = input("Enter Student ID: ")
    for student in self.students:
        if student["id"] == student_id:
            student["name"] = input("Enter New Name: ")
            student["grade"] = input("Enter New Grade: ")
            self.save_students()
            print("Student Updated")
            return

    print("Student Not Found")

```

```
def delete_student(self):  
    student_id = input("Enter Student ID: ")
```

```
    for student in self.students:  
        if student["id"] == student_id:  
            self.students.remove(student)  
            self.save_students()  
            print("Student Deleted")  
            return
```

```
    print("Student Not Found")
```

```
def list_students(self):
```

```
    if not self.students:  
        print("No Students Found")  
        return
```

```
    print("\n{:<10} {:<20} {:<10}".format(  
        "ID", "NAME", "GRADE"))
```

```
    print("-" * 40)
```

```
    for student in self.students:  
        print("{:<10} {:<20} {:<10}".format(  
            student["id"],  
            student["name"],  
            student["grade"]  
        ))
```

```
manager = StudentManager()
```

```
while True:
```

```
print("\n===== STUDENT MANAGEMENT SYSTEM =====")

print("1. Add Student")
print("2. Update Student")
print("3. Delete Student")
print("4. List Students")
print("5. Exit")

choice = input("Enter Choice: ")

if choice == "1":
    manager.add_student()

elif choice == "2":
    manager.update_student()

elif choice == "3":
    manager.delete_student()

elif choice == "4":
    manager.list_students()

elif choice == "5":
    print("Goodbye!")
    break
else:
    print("Invalid Choice")
```


14.OUTPUT

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 1

Enter Student ID: 2600

Enter Name: ALEX

Enter Grade: A+

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 1

Enter Student ID: 2601

Enter Name: BEN

Enter Grade: A

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students

5. Exit

Enter Choice: 1

Enter Student ID: 2602

Enter Name: EMMA

Enter Grade: B+

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student

2. Update Student

3. Delete Student

4. List Students

5. Exit

Enter Choice: 1

Enter Student ID: 2603

Enter Name: JACK

Enter Grade: B

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student

2. Update Student

3. Delete Student

4. List Students

5. Exit

Enter Choice: 1

Enter Student ID: 2604

Enter Name: LILY

Enter Grade: C+

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 1

Enter Student ID: 2605

Enter Name: MIYA

Enter Grade: C

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 1

Enter Student ID: 2606

Enter Name: SAM

Enter Grade: D+

Student Added Successfully

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 2

Enter Student ID: 2606

Enter New Name: RISWAN

Enter New Grade: A+

Student Updated

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student

2. Update Student

3. Delete Student

4. List Students

5. Exit

Enter Choice: 4

ID	NAME	GRADE
2600	ALEX	A+
2601	BEN	A
2602	EMMA	B+
2603	JACK	B
2604	LILY	C+
2605	MIYA	C
2606	RISWAN	A+

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student

2. Update Student

3. Delete Student

4. List Students

5. Exit

Enter Choice: 3

Enter Student ID: 2605

Student Deleted

===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. List Students
5. Exit

Enter Choice: 5

Goodbye!

15. TESTING AND VALIDATION

The application was tested extensively to ensure that all functionalities operate correctly under different scenarios.

Test Case	Expected Result	Status
Add Student	Student Added	Pass
Duplicate ID	Validation Error	Pass
Update Student	Record Updated	Pass
Delete Student	Record Removed	Pass
Display Records	Records Shown	Pass
Invalid Choice	Error Message	Pass

The results confirm that the application performs accurately and meets all specified requirements.

16.ADVANTAGES

- Easy to use interface.
- Fast record management.
- Permanent data storage.
- Prevents duplicate student IDs.
- Demonstrates Object-Oriented Programming concepts.
- Lightweight and efficient.
- Easy to maintain and modify.
- Suitable for educational institutions.

17.LIMITATIONS

- Operates through command-line interface only.
- No graphical user interface.
- Limited to local file storage.
- No database integration.
- Suitable mainly for small and medium-sized datasets.

18.FUTURE ENHANCEMENTS

The Student Management System can be enhanced with advanced features such as:

- Graphical User Interface using Tkinter.
- Database integration using MySQL or SQLite.
- Student search and filtering options.
- Attendance management module.
- Examination result processing.
- Report generation in PDF format.
- User authentication and security features.
- Cloud-based storage and backup.

19.CONCLUSION

The Student Management System developed in this project provides an efficient and reliable solution for managing student records. The system successfully automates the process of adding, updating, deleting, and displaying student information through a simple menu-driven interface. By replacing manual record management methods, the application improves accuracy, reduces administrative effort, and enables faster access to student data.

The project demonstrates the practical implementation of Python programming concepts such as Object-Oriented Programming, file handling, data validation, and JSON-based data storage. The use of classes and methods helps maintain a well-structured and organized codebase, while JSON storage ensures that student records are permanently saved and easily retrievable.

During the development of this project, important software development concepts including system analysis, system design, testing, and validation were applied successfully. The application was tested under different scenarios and produced accurate results for all operations. Validation features such as unique student ID checking further improve the reliability and integrity of the stored data.

This project has provided valuable learning experience in developing real-world applications using Python. It has strengthened understanding of programming fundamentals and demonstrated how software can be used to simplify administrative tasks. With future enhancements such as graphical user interfaces, database integration, and advanced search features, the Student Management System can be expanded into a more comprehensive educational management solution.

In conclusion, the Student Management System successfully meets its objectives and serves as an effective tool for managing student information. It is a practical, user-friendly, and scalable application that showcases the effective use of Python programming for solving real-world problems.